TeeTime Golf App

User Stories Documentation

Complete Feature Implementation Guide

Version 1.0 — December 2024

Comprehensive documentation of all implemented user stories with technical details, APIs, and business logic.

Contents

1 Introduction **2**

2 Profile & User Management **2**
2.1 US-01: View and Edit Profile . . . 2
2.2 US-02: Social Rate Score and History . . . 2
2.3 US-03: Generate Personal QR Code (Get Drafted) . . . 3
2.4 US-04: Scan QR Code (Scan to Draft) . . . 3
2.5 US-05: Complete Tee Time History . . . 4

3 Roster Management **4**
3.1 US-10: Create and Maintain Multiple Rosters . . . 4
3.2 US-11: Drag-and-Drop Between Roster and On-Deck . . . 4
3.3 US-12: Sort Roster by Multiple Criteria . . . 5
3.4 US-13: View Detailed Roster Member Profiles . . . 5
3.5 US-14: Remove or Block Users from Roster . . . 5
3.6 US-15: Notification When Added to Draft/Friends . . . 6
3.7 US-16: Add Back / Reciprocal Add Option . . . 6
3.8 US-17: Name Rosters . . . 6
3.9 US-18: Select Roster When Creating Tee Time . . . 6
3.10 US-19: Edit or Delete Saved Rosters . . . 7

4 Tee Time Creation **7**
4.1 US-20: Create Roster Invite . . . 7
4.2 US-21: Custom Response Windows . . . 7
4.3 US-22: Real-Time Status Updates with Visual Indicators . . . 8
4.4 US-23: Manually Advance to Next Roster Member . . . 8
4.5 US-24: On-Deck Selection When Roster Fails . . . 8
4.6 US-25: Create FCFS Invite . . . 9
4.7 US-26: FCFS Broadcast to Entire Roster . . . 9
4.8 US-27: First Responder Confirmation with Tie-Breaker . . . 9
4.9 US-28: FCFS Response Deadline . . . 10
4.10 US-29: Notification When Tee Time Not Filled . . . 10

5 Invite Notifications & Responses **10**
5.1 US-30: Push Notifications for New Invites . . . 10
5.2 US-31: In-App Notifications with Full Details . . . 11
5.3 US-32: Respond Yes/No to Invites . . . 11
5.4 US-33: Response Window Countdown Timer . . . 11
5.5 US-34: Notification When Accepted Player Cancels . . . 12
5.6 US-35: Automatic Re-Invite on Cancellation . . . 12
5.7 US-36: Block User from Invites . . . 12

6 Calendar & Tee Time Viewing **12**
6.1 US-37: Calendar View of Upcoming Tee Times . . . 12
6.2 US-38: Filter Tee Times by Status . . . 13
6.3 US-39: View Invited Tee Times in Calendar . . . 13
6.4 US-40: View Detailed Tee Time Information . . . 13

7 Attendance & Check-In **14**
7.1 US-41: Cancel Participation in Confirmed Tee Time . . . 14
7.2 US-42: Social Rate Penalty Warning Before Cancellation . . . 14
7.3 US-43: Pre-Tee Time Check-In Notification . . . 14
7.4 US-44: One-Tap Check-In All Players . . . 15
7.5 US-45: "Who Bailed?" No-Show Selection . . . 15
7.6 US-46: No-Show Penalty Application . . . 15

8 Technical Architecture Summary **16**
8.1 Database Schema . . . 16
8.2 Key Business Logic Components . . . 16
8.3 API Structure . . . 16

9 Appendix: API Endpoint Reference **16**

1 Introduction

This document provides a comprehensive overview of all user stories implemented in the TeeTime Golf App. Each story includes the implementation status, technical details (APIs, database tables), and business logic explanations for client comprehension.

All features listed in this document are fully implemented and tested.

2 Profile & User Management

2.1 US-01: View and Edit Profile

[IMPLEMENTED]

Story: I want the ability to view and edit my profile including past game scores, handicap, club memberships, phone number, and location, so that my information stays current.

Implementation:

- **API Endpoints:**
  - `GET /api/profile` — Retrieve current user profile
  - `PUT /api/profile` — Update profile fields
- **Database Tables:** `users`, `user_profiles`, `game_scores`, `user_club_memberships`
- **Editable Fields:** First/last name, phone, email, handicap, bio, location, home course, equipment (driver, irons, putter, ball), preferences (walk/ride, preferred tee time)
- **Logic:** Profile data split between `users` (auth info) and `user_profiles` (golf-specific data). Updates trigger `updated_at` timestamp via database trigger.

2.2 US-02: Social Rate Score and History

[IMPLEMENTED]

Story: I want the ability to view my Social Rate score and history, so that I can track my reliability reputation.

Implementation:

- **API Endpoints:**
  - `GET /api/social-rate` — Get current score and stats
  - `GET /api/social-rate/history` — Get detailed change history
- **Database:** `user_profiles.social_rate` (integer 0-100), `social_rate_logs`
- **Scoring Logic:**
  - Starting score: 100 points (no upper limit)
  - Invite accepted: +2 points
  - Decline >24 hours before: -1 point
  - Cancel <24 hours before: -5 points
  - No-show: -10 points
  - Minimum score: 0
- **Audit Trail:** Every change logged with previous score, delta, new score, reason, and tee time reference

2.3 US-03: Generate Personal QR Code (Get Drafted)

[IMPLEMENTED]

Story: I want the ability to generate my personal QR code ("Get Drafted"), so that others can easily add me to their On-Deck list.

Implementation:

- **API Endpoints:**
  - `GET /api/draft/qr` — Get or generate QR code
  - `POST /api/draft/qr/regenerate` — Invalidate old codes and create new
- **Database:** `draft_qr_codes` (code_hash, version, is_active, scanned_count)
- **Logic:**
  - QR payload contains: version, userId, random code, HMAC signature
  - Code is hashed (SHA-256) before storage — actual code never stored
  - Regeneration revokes all previous codes
  - Tracks scan count and last scanned timestamp
- **Frontend:** Uses `react-native-qrcode-svg` library, 260px size for reliable scanning

2.4 US-04: Scan QR Code (Scan to Draft)

[IMPLEMENTED]

Story: I want the ability to scan another user's QR code ("Scan to Draft") and confirmation, so that I can quickly add them to my On-Deck.

Implementation:

- **API Endpoints:**
  - `POST /api/draft/scan` — Validate QR and get user preview
  - `POST /api/draft/confirm` — Confirm adding to On-Deck
  - `GET /api/draft/history` — View who scanned your code
- **Database:** `qr_scan_history` (tracks scanner, scanned user, timestamp, was_added)
- **Validation:**
  - Verify HMAC signature matches
  - Check code is active and not revoked
  - Prevent self-scanning
  - Check if already in roster
- **Frontend:** Uses `expo-camera` with barcode scanning, 2-second cooldown between scans, animated scan line overlay

2.5 US-05: Complete Tee Time History

[IMPLEMENTED]

Story: I want the ability to view my complete tee time history (past and upcoming), so that I can track my golf activity.

Implementation:

- **API:** `GET /api/tee-times/history?filter=<status>`
- **Filters:** all, upcoming, completed, cancelled, no_show
- **Response Data:** Course name, date/time, status, user role (organizer/participant), attendance status, game score, participant count
- **Logic:** Combines tee times where user is creator OR participant, sorted by date descending

3 Roster Management

3.1 US-10: Create and Maintain Multiple Rosters

[IMPLEMENTED]

Story: I want the ability to create and maintain multiple rosters of golf buddies, so that I can organize my regular playing partners into selectable groups.

Implementation:

- **API Endpoints:**
  - `GET /api/roster/groups` — List all groups
  - `POST /api/roster/groups` — Create new group
  - `PUT /api/roster/groups/:id` — Update group
  - `DELETE /api/roster/groups/:id` — Delete group
- **Database:** `roster_groups`, `roster_group_members`
- **Group Properties:** name, description, color (hex), icon, auto_invite flag, max_members, sort_preference
- **Logic:** Many-to-many relationship allows members to belong to multiple groups

3.2 US-11: Drag-and-Drop Between Roster and On-Deck

[IMPLEMENTED]

Story: I want the ability to drag-and-drop golfers between "Roster" and "On Deck" sections, so that I can prioritize my preferred playing partners.

Implementation:

- **API Endpoints:**
  - `PUT /api/roster/reorder` — Update positions after drag
  - `PUT /api/roster/:id/move` — Move between sections
- **Database:** `roster.position` (integer), `roster.is_on_deck` (boolean)
- **Definitions:**
  - **Roster:** Players who will be auto-invited (is_on_deck = false)
  - **On-Deck:** Reserve players manually selected when roster fails to fill (is_on_deck = true)
- **Frontend:** React Native Draggable FlatList with smooth animations

3.3 US-12: Sort Roster by Multiple Criteria

[IMPLEMENTED]

Story: I want the ability to sort my roster by Social Rate, Distance to Course, Handicap, or Club Membership, so that I can make strategic invite decisions.

Implementation:

- **API:** `GET /api/roster?sortBy=<option>`
- **Sort Options:**
  - `position` — Manual drag order (default)
  - `socialRate` — Reliability score (descending)
  - `handicap` — Skill level (ascending)
  - `name` — Alphabetical
  - `distance` — Proximity to course
  - `membership` — Club membership status
- **Logic:** Dynamic SQL ORDER BY based on parameter

3.4 US-13: View Detailed Roster Member Profiles

[IMPLEMENTED]

Story: I want the ability to view detailed profiles of roster members including Social Rate, history, handicap, memberships, and waiver wire, so that I have complete information.

Implementation:

- **API:** `GET /api/roster/:id/profile`
- **Profile Data:**
  - Contact info: name, username, avatar, bio
  - Stats: handicap, social rate, rounds played, average score
  - History: times played together, invite acceptance rate, last played date
  - Preferences: location, home course, walk/ride, preferred tee time
  - Equipment: driver, irons, putter, ball
- **Database:** Joins `roster`, `users`, `user_profiles`

3.5 US-14: Remove or Block Users from Roster

[IMPLEMENTED]

Story: I want the ability to remove or block users from my roster, so that I can maintain control over my connections.

Implementation:

- **API Endpoints:**
  - `DELETE /api/roster/:id` — Remove from roster
  - `POST /api/friends/block/:userId` — Block user
- **Database:** `blocked_users` table
- **Block Logic:**

- Removes from both users' rosters
- Prevents future friend requests
- Hides user from discovery
- Constraint prevents self-blocking

3.6 US-15: Notification When Added to Draft/Friends

[IMPLEMENTED]

Story: I want the ability to receive notifications when someone adds me to their draft/friends, so that I'm aware of the connection.

Implementation:

- **Service:** `rosterNotificationService.js`
- **Database:** `roster_notifications`, `notifications`
- **Notification Data:** Owner name, avatar, group name (if applicable)
- **Logic:** Triggered when `rosterRepository.addToRoster()` is called with a registered user

3.7 US-16: Add Back / Reciprocal Add Option

[IMPLEMENTED]

Story: I want the ability to receive a "Add Back" or reciprocal add option in the notification, so that I can easily add them to my draft/friends if I want to.

Implementation:

- **Notification Action:** "Add Back" button in notification
- **Logic:** Tapping "Add Back" calls `POST /api/roster` with the owner's userId
- **UI:** Action button visible in notification card, disabled after action taken

3.8 US-17: Name Rosters

[IMPLEMENTED]

Story: I want the ability to name my rosters (e.g., 'Weekend Crew'), so that I can identify them easily.

Implementation:

- **API:** `POST /api/roster/groups` with `name` field
- **Database:** `roster_groups.name` (VARCHAR 100)
- **Constraint:** Unique name per user (user_id, name)

3.9 US-18: Select Roster When Creating Tee Time

[IMPLEMENTED]

Story: I want the ability to select which roster to use when creating a tee time invite, so that I can send invites to the appropriate group.

Implementation:

- **UI Flow:** Tee time creation wizard includes roster/group selection step
- **API:** `GET /api/roster/groups/:id/members` to fetch group members
- **Logic:** Selected group members become the invite list, respecting group's sort preference

3.10 US-19: Edit or Delete Saved Rosters

[IMPLEMENTED]

Story: I want the ability to edit or delete saved rosters, so that I can maintain my roster groups over time.

Implementation:

- **API Endpoints:**
  - `PUT /api/roster/groups/:id` — Update group properties
  - `DELETE /api/roster/groups/:id` — Delete group
- **Delete Logic:** Group deletion sets `group_id = NULL` on roster entries (members not deleted)

4 Tee Time Creation

4.1 US-20: Create Roster Invite

[IMPLEMENTED]

Story: I want the ability to create a Roster invite by selecting date, time, course (with autocomplete), and group size (2/3/4), so that I can organize rounds by the roster group selected and a custom order if desired.

Implementation:

- **API Endpoints:**
  - `POST /api/tee-times` — Create tee time
  - `POST /api/invites` with `inviteType: 'roster'`
- **Course Autocomplete:** `GET /api/courses/search?q=<query>` — US golf course database
- **Group Size:** `max_players` field (2, 3, or 4)
- **Logic:** Creator selects roster members in desired order; position determines invite priority

4.2 US-21: Custom Response Windows

[IMPLEMENTED]

Story: I want the ability to set custom response windows (1-24 hours) for each roster position, so that I can control the invitation timeline.

Implementation:

- **API:** `POST /api/invites` with `responseWindow` parameter (hours)
- **Database:** `invites.expires_at` (TIMESTAMPTZ)
- **Options:** 2 minutes (test), 1 hour, 2 hours, 4 hours, 8 hours, 12 hours, 24 hours
- **Logic:** All invitees receive same `expires_at` timestamp; priority determined by invite position

4.3 US-22: Real-Time Status Updates with Visual Indicators

[IMPLEMENTED]

Story: I want the ability to see real-time status updates on roster invites with visual indicators (green check for yes, arrow for waiting, decline icon, timeout icon), so that I can track responses at a glance.

Implementation:

- **API:** `GET /api/tee-times/:id` returns invite statuses
- **Status Icons:**
  - Green checkmark: Accepted
  - Clock/arrow: Pending (waiting for response)
  - X icon: Declined
  - Timer icon: Expired/timed out
  - Queue icon: Waitlisted
- **Frontend:** Polling or real-time updates to refresh invite statuses

4.4 US-23: Manually Advance to Next Roster Member

[IMPLEMENTED]

Story: I want the ability to manually advance to the next roster member if someone declines before the window expires, so that I can fill spots faster.

Implementation:

- **API:** `POST /api/tee-times/:id/invites/:inviteId/skip`
- **Logic:**
  - Marks current invite as 'skipped'
  - Activates next pending invite in position order
  - Returns updated invite list
- **UI:** "Skip" button on pending invite cards

4.5 US-24: On-Deck Selection When Roster Fails

[IMPLEMENTED]

Story: I want the ability to receive a notification that my Tee Time cannot be filled by my roster and asked if I would like to select players from my On-Deck players to fill the remaining slots.

Implementation:

- **Service:** `fcfsDeadlineService.js` (cron job)
- **Trigger:** When all roster invites expire/decline and spots remain unfilled
- **Notification:** "Your tee time has unfilled spots. Select from On-Deck?"
- **API:** `POST /api/tee-times/:id/on-deck-invites`
- **Logic:** On-Deck players receive FCFS invites; first to accept gets spot

4.6 US-25: Create FCFS Invite

[IMPLEMENTED]

Story: I want the ability to create a First Come First Served invite by selecting date, time, course (with autocomplete), and group size (2/3/4), so that I can quickly organize my round.

Implementation:

- **API:** `POST /api/invites` with `inviteType: 'fcfs'`
- **Logic:**
  - All invitees notified simultaneously
  - First to accept gets confirmed spot
  - When full, remaining invites moved to waitlist
  - Atomic acceptance prevents race conditions

4.7 US-26: FCFS Broadcast to Entire Roster

[IMPLEMENTED]

Story: I want the ability to create a First Come First Serve invite that broadcasts to my entire roster, so that I can quickly fill spots for spontaneous rounds.

Implementation:

- **API:** `POST /api/invites/from-roster` with `teeTimeId`
- **Logic:** Fetches all roster members (excluding On-Deck) and creates FCFS invites for all
- **UI:** "Invite Entire Roster" button in FCFS creation flow

4.8 US-27: First Responder Confirmation with Tie-Breaker

[IMPLEMENTED]

Story: I want the ability to see who responds first and is automatically confirmed, so that spots are filled efficiently. If we need to determine tie-breaker criteria, the person who is higher in my roster should get priority.

Implementation:

- **Repository:** `inviteRepository.acceptInviteAtomic()`
- **Tie-Breaker Logic:**
  - 500ms delay to collect near-simultaneous accepts
  - Database row locking: `SELECT ... FOR UPDATE`
  - For FCFS: Roster position determines priority
  - For Roster invites: Invite position (order invited) determines priority
  - Lower position number = higher priority
  - Winner added as participant; loser moved to waitlist
- **Transaction:** Entire operation in single database transaction for atomicity

4.9 US-28: FCFS Response Deadline

[IMPLEMENTED]

Story: I want the ability to set a deadline for FCFS responses, so that I can finalize plans by a certain time.

Implementation:

- **API:** `POST /api/tee-times/:id/fcfs-deadline`
- **Quick Presets:** 2 minutes, 5 minutes, 1 hour, 24 hours
- **Logic:** Updates `expires_at` for all pending invites on the tee time
- **Expiration:** Cron job marks expired invites and notifies organizer

4.10 US-29: Notification When Tee Time Not Filled

[IMPLEMENTED]

Story: I want the ability to have the app notify me if my tee time is not filled by the expiration date, and prompt me to select from my on-deck to fill.

Implementation:

- **Service:** `fcfsDeadlineService.js`
- **Check Logic:**
  - Runs periodically (cron)
  - Finds tee times with expired invites and unfilled spots
  - Checks tee time status is 'pending' or 'open'
- **Notification:** Push notification to organizer with action to select On-Deck players
- **On-Deck Flow:** Selected players receive FCFS invites

5 Invite Notifications & Responses

5.1 US-30: Push Notifications for New Invites

[IMPLEMENTED]

Story: I want the ability to receive push notifications for new invites, so that I can respond quickly.

Implementation:

- **Service:** `notificationService.notifyTeeTimeInvite()`
- **Database:** `notifications`, `devices.push_token`
- **Notification Content:** Inviter name, course, date, time
- **Logic:** Triggered when invite is created; sends to all registered devices

5.2 US-31: In-App Notifications with Full Details

[IMPLEMENTED]

Story: I want the ability to receive in-app notifications for new invites with full details (who, when, where, group size), so that I have context to make decisions.

Implementation:

- **API:** `GET /api/notifications`
- **Notification Data:**
  - `inviterName`, `inviterAvatar`
  - `courseName`, `date`, `time`
  - `teeTimeId` for navigation
  - `maxPlayers` (group size)
- **UI:** Notification bell with unread count badge

5.3 US-32: Respond Yes/No to Invites

[IMPLEMENTED]

Story: I want the ability to respond Yes/No to invites within the response window, so that I can confirm or decline participation.

Implementation:

- **API:** `PUT /api/invites/:id/respond`
- **Request Body:** {`response: 'accepted' | 'declined', message?: string`}
- **Logic:**
  - Validates invite is still pending and not expired
  - For 'accepted': Uses atomic acceptance with tie-breaker
  - For 'declined': Updates status, notifies organizer
  - Records response in `invite_responses` table

5.4 US-33: Response Window Countdown Timer

[IMPLEMENTED]

Story: I want the ability to view the response window countdown timer, so that I know how long I have to decide.

Implementation:

- **Data:** `invites.expires_at` returned in invite details
- **Frontend:** Real-time countdown timer (HH:MM:SS format)
- **Logic:**
  - Updates every second using `setInterval`
  - Shows "Urgent" state (red) when <1 hour remaining
  - Disables response buttons when expired
- **UI:** Prominent timer display on invite cards

5.5 US-34: Notification When Accepted Player Cancels

[IMPLEMENTED]

Story: I want the ability to receive a notification if a player that has accepted an invite cancels.

Implementation:

- **Trigger:** `POST /api/tee-times/:id/cancel-participation`
- **Notification:** Sent to organizer with canceling player's name
- **Content:** "{PlayerName} has cancelled their spot for your tee time at {Course}"

5.6 US-35: Automatic Re-Invite on Cancellation

[IMPLEMENTED]

Story: I want the ability to have the app automatically send a new invite based on the selected invite method (Roster or FCFS) if a player cancels after accepting an invite.

Implementation:

- **Logic by Invite Type:**
  - **Roster:** Continues sending invites in position order to next pending
  - **FCFS:** Notifies waitlisted players and those who haven't responded: "A spot has opened up in {User's} Tee Time on {Date/time} at {Course}. First to Respond is in!"
- **API:** `inviteRepository.reactivateWaitlistInvites()`
- **Notification:** Push notification to eligible players

5.7 US-36: Block User from Invites

[IMPLEMENTED]

Story: I want the ability to block a user if I no longer want to receive invites from that user.

Implementation:

- **API:** `POST /api/friends/block/:userId`
- **Database:** `blocked_users` table
- **Effects:**
  - Blocked user cannot send invites to blocker
  - Removes from both users' rosters
  - Hides from discovery/search
  - Prevents friend requests

6 Calendar & Tee Time Viewing

6.1 US-37: Calendar View of Upcoming Tee Times

[IMPLEMENTED]

Story: I want the ability to view a calendar of all my upcoming tee times, so that I can see my golf schedule at a glance.

Implementation:

- **API:** `GET /api/tee-times/calendar?start=<date>&end=<date>`
- **Response:** Object keyed by date with array of tee times per day
- **Logic:** Returns tee times where user is creator OR confirmed participant
- **UI:** Calendar component with dots indicating tee time days; tap to see details

6.2 US-38: Filter Tee Times by Status

[IMPLEMENTED]

Story: I want the ability to filter tee times by status (pending, confirmed, completed), so that I can organize my schedule.

Implementation:

- **API:** `GET /api/tee-times?status=<status>`
- **Status Options:** pending, confirmed, in_progress, completed, cancelled
- **UI:** Filter tabs or dropdown in tee times list

6.3 US-39: View Invited Tee Times in Calendar

[IMPLEMENTED]

Story: I want the ability to see tee times I have been invited to in the calendar so that I can respond Yes or No to them.

Implementation:

- **API:** `GET /api/invites/pending`
- **Calendar Integration:** Pending invites shown with different color/indicator
- **UI:** Tap on invited tee time opens response modal with Yes/No buttons
- **Logic:** Combines owned tee times + pending invites in calendar view

6.4 US-40: View Detailed Tee Time Information

[IMPLEMENTED]

Story: I want the ability to view detailed information for each tee time (participants, course, time, invite type), so that I have complete context.

Implementation:

- **API:** `GET /api/tee-times/:id`
- **Response Data:**
  - Course: name, location, course_id
  - Time: tee_date, tee_time, holes (9/18)
  - Participants: list with name, avatar, status, isCreator
  - Invites: list with status, position, expires_at
  - Settings: invite_type, max_players, response_window
  - Status: pending, confirmed, in_progress, completed, cancelled
- **UI:** TeeTimeDetailScreen with hero header, participant cards, invite status grid

7 Attendance & Check-In

7.1 US-41: Cancel Participation in Confirmed Tee Time

[IMPLEMENTED]

Story: I want the ability to cancel my participation in a confirmed tee time, so that I can manage schedule conflicts.

Implementation:

- **API:** `POST /api/tee-times/:id/cancel-participation`
- **Logic:**
  - Updates participant status to 'cancelled'
  - Calculates Social Rate penalty based on timing
  - Notifies organizer of cancellation
  - Triggers re-invite flow (Roster or FCFS)

7.2 US-42: Social Rate Penalty Warning Before Cancellation

[IMPLEMENTED]

Story: I want the ability to see the Social Rate penalty with a pop-up stating "You will lose {points} points if you cancel your Tee Time now. Confirm cancellation? [Yes] [No]" before canceling, so that I understand the accountability consequence.

Implementation:

- **Logic:** Calculate penalty based on time until tee time:
  - >24 hours: -1 point (minor penalty)
  - <24 hours: -5 points (late cancellation)
- **UI:** Confirmation modal showing exact point loss before action
- **Response:** Returns `penaltyApplied: true/false` and points lost

7.3 US-43: Pre-Tee Time Check-In Notification

[IMPLEMENTED]

Story: I want the ability as the organizer to receive a push notification 5 minutes before my scheduled tee time asking, "Is Everyone Here?", so that I can quickly confirm attendance.

Implementation:

- **Database:** `tee_times.check_in_reminder_sent` (boolean flag)
- **Service:** Scheduled job checks for tee times 5 minutes away
- **Notification:** "Is Everyone Here?" with action buttons
- **Logic:** Only sent once per tee time; flag prevents duplicates

7.4 US-44: One-Tap Check-In All Players

[IMPLEMENTED]

Story: I want the ability as the organizer to select "Yes" to automatically check in all players with no further action required, so that attendance confirmation is quick and effortless.

Implementation:

- **API:** `POST /api/tee-times/:id/check-in/all`
- **Logic:**
  - Updates all participants' attendance status to 'present'
  - Sets `checked_in_at` timestamp
  - Records `checked_in_by` (organizer)
  - Marks tee time check-in as completed
- **Response:** Count of players checked in

7.5 US-45: "Who Bailed?" No-Show Selection

[IMPLEMENTED]

Story: I want the ability as the organizer to select "No" which opens a "Who Bailed?" page showing all confirmed players as checkboxes, so that I can select only the no-show players who will receive the -10 point penalty.

Implementation:

- **API:** `GET /api/tee-times/:id/attendance` — Get participant list
- **UI:** Checkbox list of all confirmed participants
- **Logic:**
  - Organizer selects players who didn't show
  - Unchecked players marked as 'present'
  - Checked players marked as 'no_show'

7.6 US-46: No-Show Penalty Application

[IMPLEMENTED]

Story: I want the ability for the organizer to mark my No-Show during check-in, so the system applies a no-show penalty (-10 points) if I do not cancel my Tee Time invite.

Implementation:

- **API:** `POST /api/tee-times/:id/check-in/:attendanceId`
- **Request Body:** {`status: 'no_show'`}
- **Logic:**
  - Updates `tee_time_attendance.status` to 'no_show'
  - Sets `marked_no_show_at` timestamp
  - Applies -10 point Social Rate penalty
  - Creates entry in `social_rate_logs` with reason 'no_show'
  - Sets `penalty_applied = true`
- **Database:** `tee_time_attendance`, `social_rate_logs`

8 Technical Architecture Summary

8.1 Database Schema

The application uses PostgreSQL with 32 tables organized into logical groups:

- **Core Tables:** users, user_profiles, devices, sessions, otp_requests, magic_links
- **Social/Roster:** roster, roster_groups, roster_group_members, friendships, friend_requests, blocked_users
- **Tee Times:** tee_times, tee_time_participants, invites, invite_responses
- **Accountability:** social_rate_logs, tee_time_attendance
- **Supporting:** notifications, golf_courses, draft_qr_codes, qr_scan_history

8.2 Key Business Logic Components

- **Atomic Invite Acceptance:** Database row locking with 500ms delay for tie-breaker resolution
- **Social Rate System:** Automatic penalty/reward calculation with full audit trail
- **QR Code Security:** HMAC-signed payloads, hashed storage, revocation support
- **Invite Expiration:** Cron-based monitoring with automatic status updates and notifications
- **Attendance Tracking:** Pre-tee time reminders, one-tap check-in, no-show penalty automation

8.3 API Structure

All APIs follow RESTful conventions:

- **Authentication:** JWT tokens with automatic refresh
- **Base URL:** `/api/`
- **Modules:** auth, profile, roster, tee-times, invites, friends, notifications, courses, social-rate, attendance, draft

9 Appendix: API Endpoint Reference

Endpoint	Description	User Story
GET /api/profile	Get user profile	US-01
PUT /api/profile	Update profile	US-01
GET /api/social-rate	Get social rate score	US-02
GET /api/social-rate/history	Get score history	US-02
GET /api/draft/qr	Get QR code	US-03
POST /api/draft/scan	Scan QR code	US-04
POST /api/draft/confirm	Confirm draft	US-04

Endpoint	Description	User Story
GET /api/tee-times/history	Get tee time history	US-05
GET /api/roster	Get roster	US-10
PUT /api/roster/reorder	Reorder roster	US-11
PUT /api/roster/:id/move	Move member	US-11
GET /api/roster/groups	Get groups	US-10, US-17
POST /api/roster/groups	Create group	US-10, US-17
GET /api/roster/:id/profile	Get member profile	US-13
POST /api/friends/block/:id	Block user	US-14, US-36
POST /api/tee-times	Create tee time	US-20, US-25
POST /api/invites	Create invites	US-20, US-21, US-25
POST /api/tee-times/:id/skip	Skip invite	US-23
POST /api/tee-times/:id/on-deck-invites	On-deck invites	US-24, US-29
POST /api/tee-times/:id/fcfs-deadline	Set deadline	US-28
GET /api/notifications	Get notifications	US-30, US-31
PUT /api/invites/:id/respond	Respond to invite	US-32
GET /api/invites/pending	Get pending invites	US-33, US-39
GET /api/tee-times/calendar	Get calendar	US-37
GET /api/tee-times?status=	Filter tee times	US-38
GET /api/tee-times/:id	Get tee time details	US-40
POST /api/tee-times/:id/cancel-participation	Cancel participation	US-41, US-42
POST /api/tee-times/:id/check-in/all	Check in all	US-44
GET /api/tee-times/:id/attendance	Get attendance	US-45
POST /api/tee-times/:id/check-in/:id	Update attendance	US-46